1.1 — API Basics: Requests, Responses, Clients, and Servers

An API, or application programming interface, lets one piece of software ask another for data or an action. Think of it like ordering at a restaurant: one side places an order, and the other sends back the result.

Web APIs follow a request/response pattern. The **client** sends the request. The **server** receives it and sends back a response. The request says where to go, what to do, and sometimes includes data. The response returns a result, often with a status code and data.

Part	Meaning
Request	A message sent to ask for data or trigger an action
Response	The server's answer, including status information and often data

A browser, mobile app, or tool like Insomnia can act as the client. A Spring Boot application often acts as the server. If the terms feel unfamiliar, keep the core idea in mind: one side asks, and the other answers.

TIP
When you hear “API,” think of a structured conversation: one side asks, and the other side answers.

1.2 — Endpoints and Resources

To understand a request, first identify what it is targeting. In a REST API, that target is described by a **resource** and an **endpoint**.

A **resource** is the type of data the API manages, such as recipes, users, or products. An **endpoint** is the URL used to access that resource.

For example, in a recipe API, `recipes` is the resource. `/recipes` points to the full collection of recipes we have in our database. `/recipes/12` points to one specific recipe, namely recipe #12.

- `/recipes` refers to the recipes resource as a collection
- `/recipes/12` refers to one specific recipe in that collection

So before we start thinking about verbs, we'll start by asking:

- What resource is this endpoint about?
- Is this endpoint referring to the whole collection or one specific item?

Once that is clear, the next question is what you want to do with that endpoint.

1.3 — HTTP Verbs and What You Want to Do

Start with the common HTTP verbs first. These verbs tell you the kind of action the request is trying to perform.

Here are the verbs you will see in a REST API:

CRUD Operation	Common HTTP Verb
Create	`POST`
Read	`GET`
Update	`PUT` or `PATCH`
Delete	`DELETE`

Now add the endpoint. The endpoint tells you what the request is about, and the verb gives context by telling you what you want to do with that endpoint.

For example, start with these two endpoints:

- `/recipes` — the full recipes collection
- `/recipes/12` — one specific recipe

Once you combine the endpoint with the verb, the request becomes much clearer:

- `GET /recipes` retrieves the full collection of recipes
- `GET /recipes/12` retrieves recipe #12
- `POST /recipes` creates a new recipe in the collection
- `PUT /recipes/12` replaces or fully updates recipe #12
- `PATCH /recipes/12` partially updates recipe #12
- `DELETE /recipes/12` deletes recipe #12

In REST, the endpoint tells you what the request is about, and the verb tells you what to do with it.

In a REST API, URLs usually use **nouns**, not verbs. For example, a RESTful API typically uses `/recipes`, not `/getRecipes`. The URL identifies the resource. The HTTP verb communicates the action.

1.4 — Decoding the Full URL

Once you understand the endpoint and the verb, the next step is reading the full URL. The full URL shows where the request is going and any extra instructions attached to it.
Example:
`https://api.example.com/api/recipes/12?includeIngredients=true`

Base URL

`https://api.example.com/api`
The **base URL** identifies the server that hosts the API.

Path

`/recipes/12`
The **path** identifies the resource and, here, one specific item in it.

Path Parameter

12
A **path parameter** is a value inside the path that identifies one specific item.

- `/recipes` refers to the recipes collection
- `/recipes/12` refers specifically to recipe #12

Query Parameters

`?includeIngredients=true`
Query parameters come after a question mark and add filters or extra instructions.

- `/recipes?category=dessert` filters results so we only get back desserts
- `/recipes?maxCookTime=30` filters results by cook time
- `/recipes/12?includeIngredients=true` asks for recipe 12 and includes ingredient details

1.5 — JSON Basics

APIs commonly use **JSON** to structure request and response data. JSON is lightweight, readable text, which is why you will see it often in both API requests and API responses.

JSON

```
{"id": 12, "name": "Chocolate Cake"}
```

A JSON object uses curly braces, an array uses square brackets, and fields are written as key/value pairs such as `"name": "Chocolate Cake"`.

When you send JSON to an API, it usually goes in the **request body**. When the API returns JSON, it usually appears in the **response body**.

TIP

JSON is just text, but its structure matters. Pay close attention to braces, brackets, commas, and quotation marks.

1.6 — HTTP Status Codes

HTTP status codes tell you what happened after the server handled a request. You do not need to memorize many at first. Just learn the common ones you will see most often.

Code	Meaning	When You Might See It
`200 OK`	Request succeeded	You successfully retrieved data
`201 Created`	A new resource was created	You sent a POST request that added a new item
`204 No Content`	Request succeeded with no response body	You deleted an item or updated something without returning data
`400 Bad Request`	The request was invalid	A parameter or JSON body was malformed or missing required data
`404 Not Found`	The resource was not found	The ID or path did not match anything on the server
`500 Internal Server Error`	The server failed while handling the request	Something went wrong in the application

1.7 — Exploring and Testing APIs with Swagger and Insomnia

Swagger and Insomnia let you explore APIs without building a full client application.

In Swagger, you can inspect endpoints, review inputs, send a request, and inspect the response.

In Insomnia, you build a request by entering the URL, choosing the HTTP verb, and optionally adding JSON in the request body.

Your instructor will show you how to use Insomnia.

If a request fails, first check the URL, HTTP verb, parameters, and JSON structure.

1.8 — Mini Workshop 1.1: Exploring the Recipe API

In this mini workshop, you will explore a working REST API by sending real requests and inspecting real responses. The goal is to get comfortable entering endpoints and noticing what comes back.

As you work through the workshop, use this process each time:

1. Predict what the request is asking for
2. Try the request in Swagger or Insomnia
3. Compare what you expected with what actually came back

Before You Begin

1. Open the project in **IntelliJ**
2. Run **SpringDemoApplication**
3. Wait until the console shows that the application has started
4. Open one of the following tools:
    - **Swagger:** `http://localhost:8080/swagger-ui/index.html`
    - **Insomnia:** use base URL `http://localhost:8080`
5. Confirm that you can see the Recipe API endpoints under `/api/recipes`
6. Your instructor will show you how to make requests using swagger or Insomnia. Make the request and see what you get back.

Note: On first run, the app loads sample recipe data. If your list looks empty or incorrect, stop the app, delete the local database file, and restart.

Part 1 — GET Requests: Reading Data

GET requests read data. They do not change anything on the server.

1.8.1 — Get the Full Recipe Collection

Send this request: `GET /api/recipes`

Prompt	Your answer
What do you think `GET /api/recipes` is asking for?	
What actually came back?	
Make note of the status code you saw.	

1.8.2 — Use Query Parameters to Filter Results

Try each request and compare the results.

Request	What do you think it is asking for?	What actually came back?
`GET /api/recipes?name=soup`		

1.8.3 — Use a Path Parameter to Get One Recipe

Request	What do you think this request is asking for?	What actually came back?
`GET /api/recipes/1`		

Part 2 — POST Requests: Creating Data

POST requests create new data. They usually include a JSON request body.

1.8.4 — Create a New Recipe

Send this request: `POST /api/recipes`

Use this example body:

JSON Request Body

```
{
"name": "Grilled Cheese",
"ingredients": "bread, cheddar, butter",
"instructions": "Butter bread, add cheese, grill until melted."
}
```

Prompt	Your answer
What do you think this request is trying to do?	
What came back in the response?	
Make note of the status code you saw.	

Check	Your answer

Part 3 — Update One Recipe

PUT and PATCH both update data, but they do not work the same way.

1.8.5 — Change One Field with PATCH

Send this request: `PATCH /api/recipes/{id}`

Use the recipe you created earlier if possible.

JSON Request Body

```
{
"name": "Super Grilled Cheese"
}
```

Prompt	Your answer
What do you think this request is trying to change?	
What actually changed?	
Make note of the status code you saw.	

1.8.6 — Delete a Recipe

Send this request: `DELETE /api/recipes/{your-test-id}`

Prompt	Your answer
What do you think this request is asking the API to do?	
What actually happened?	
Make note of the status code you saw.	

DELETE removes a resource.

Send this request: `DELETE /api/recipes/{your-test-id}`